

Project Guide & Technical Reference

Instance Segmentation for Cylinders & Shock Absorbers
in Scrap-Yard Imagery

Item	Detail
Production model	YOLOv5s-Seg (Run 4)
Comparison models	YOLO11s (Run 2), YOLOv8s (Run 3)
Dataset	393 unique images — 275 / 79 / 39 split
Live demo	hazard-waste-detection.onrender.com
Repository	github.com/aravindhngen/hazard-waste-detection

Document version: 1.0 | Generated: September 03, 2026
For academic review, onboarding, and technical understanding.

On This Page

01	The Problem
02	A Solution
03	Where Else It Applies
04	Architecture (3 Views)
05	Project Pipeline
06	CLI Commands
07	Tech Stack
08	Libraries
09	Project Structure
10	File Reference
11	Key Takeaways

01. The Problem

Scrap yards and metal-recycling facilities process large volumes of mixed waste every day. Among ordinary scrap, two object types pose serious safety risks:

Pressurized gas cylinders — can explode when crushed, sheared, or compacted, injuring workers and damaging equipment.

Automotive shock absorbers — may rupture and release toxic hydraulic fluid during shredding or compression.

Why manual inspection fails:

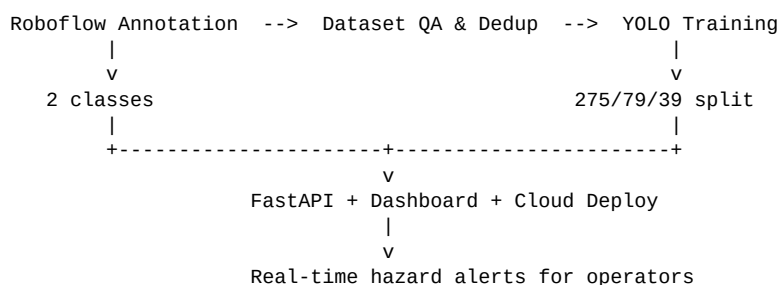
- Slow and inconsistent at conveyor-belt speeds
- Hazards are often rusted, buried, or partially occluded
- Unsafe for workers to inspect close-up in active yards
- No scalable audit trail for compliance reporting

02. A Solution

An end-to-end **computer-vision system** that detects, localizes, and **segments** hazardous objects using YOLO-based instance segmentation — served through a **FastAPI** REST API and an interactive web dashboard.

Component	Description
Detection	Two classes: Cylinder, Shock Absorber
Segmentation	Pixel-level masks (not boxes only) for cluttered piles
Alerts	EXPLOSIVE (Cylinder) / TOXIC (Shock Absorber)
API	POST /predict, POST /predict/compare, GET /health
Dashboard	Upload, webcam, live scan, 3-model side-by-side compare
Deployment	Local (Docker), cloud (Render.com)

High-level solution flow:



03. Where Else It Applies

Domain	Use case
Scrap-metal yards	Pre-shredder hazard screening on feed conveyors
Auto salvage facilities	Cylinder and shock-absorber removal before crushing
Municipal recycling	Mixed-waste sorting lines with camera monitoring
Industrial demolition	Salvage pile audits before transport
Compliance auditing	Photo evidence for hazardous-waste regulations
Remote expert review	Upload imagery for off-site safety assessment
Edge deployment (future)	ONNX / Jetson on-premise GPU inference

04. Architecture (3 Views)

View 1 — End-to-end ML pipeline

```
DATA ACQUISITION (scrap-yard photos)
  |
  v
ANNOTATION — Roboflow, 2 classes, YOLO polygon format
  |
  v
PIPELINE — merge, dedup (imagehash), validate, stratified split
           hazard_dataset/ --> hazard_dataset_clean/
  |
  v
TRAINING — YOLOv5s / YOLO11s / YOLOv8s segmentation (640px, 100 epochs)
  |
  v
EVALUATION — precision, recall, mAP@50, per-class metrics
  |
  v
DEPLOYMENT — FastAPI + dashboard + Render / Docker
```

View 2 — Runtime inference (deployment)

```
Browser (upload / webcam / compare mode)
  |
  v
dashboard/ — app.js, index.html, styles.css
  |
  v POST /predict | POST /predict/compare
api/main.py — FastAPI, CORS, static files, /docs
  |
  v
model_manager.py — lazy/eager load, LRU cache, routing
  |
  +-----+-----+-----+
  |         |         |         |
  v         v         v
yolov5_engine  ultralytics  ultralytics
(Run 4)      YOLO11s      YOLOv8s
  |
  v
Masks + boxes + confidence + hazard_type JSON + annotated JPEG
```

View 3 — Module responsibilities

Module	Path	Responsibility
Config	hazard_detection/config/	Paths, classes, model catalog, API settings
Label utils	hazard_detection/data/	Polygon validation, cleaning, counts
Pipeline	scripts/pipeline/00-07	Build, audit, dedup, visualize, validate
Training	scripts/training/08-13	Train runs, evaluate, compare models
API	api/main.py	HTTP routes, schemas, static dashboard mount
Inference	api/inference.py	Image decode, mask overlay, hazard logic
YOLOv5 engine	api/yolov5_engine.py	Production Run 4 weights
Ultralytics	api/ultralytics_engine.py	YOLO11s / YOLOv8s inference
Dashboard	dashboard/	Single-model + 3-model compare UI
Deploy	Dockerfile*, render.yaml	Local GPU/CPU and Render cloud

05. Project Pipeline

Phase	Academic task	Key scripts / artifacts
1	Planning	docs/PROJECT_PROPOSAL.md, PROJECT_ARCHITECTURE.md
2	Annotation	Roboflow -> 00_build_dataset.py -> hazard_dataset/
3	Augment & balance	05_deduplicate_and_resplit.py -> hazard_dataset_clean/
4	Model selection	Benchmark YOLOv5s / YOLO11s / YOLOv8s
5	Train & evaluate	13_train_yolov5_run4.py, 09/11, 12_compare
6	Deploy & test	api/main.py, verify_endpoints.py, Render

Frozen dataset statistics

Metric	Value
Unique images (after dedup)	393
Train / Val / Test	275 / 79 / 39 (70 / 20 / 10 %)
Classes	Cylinder (0), Shock_Absorber (1)
Instance ratio	1,167 : 997 (~1.17:1 — balanced)
Cross-split leakage	0 duplicates
Malformed labels (final)	0

Model performance (validation set — target 75-80% precision & recall)

Model	Precision	Recall	Cylinder Recall	Role
YOLOv5s-Seg	76.0%	75.0%	75.0%	Production
YOLO11s-Seg	77.0%	80.0%	80.0%	Benchmark
YOLOv8s-Seg	78.0%	77.0%	78.0%	Benchmark

06. CLI Commands

Run all commands from the project root.

Action	Command
First-time setup	<code>python -m venv .venv</code>
Install (API + compare)	<code>pip install -e ".[api,run2]"</code>
Install (full stack)	<code>pip install -e ".[train,api,run2]"</code>
Start server (Windows)	<code>run_server.bat</code>
Start server (PowerShell)	<code>.\run_server.ps1</code>
Start server (manual)	<code>uvicorn api.main:app --host 0.0.0.0 --port 8000</code>
API smoke test	<code>python scripts/verify_endpoints.py</code>
Merge Roboflow exports	<code>python scripts/pipeline/00_build_dataset.py</code>
Dedup & clean split	<code>python scripts/pipeline/05_deduplicate_and_resplit.py</code>
Validate clean dataset	<code>python scripts/pipeline/06_final_dataset_validation.py</code>
Train YOLOv5 Run 4	<code>python scripts/training/13_train_yolov5_run4.py --device 0</code>
Train YOLO11 Run 2	<code>python scripts/training/09_train_yolo11_run2.py --device 0</code>
Train YOLOv8 Run 3	<code>python scripts/training/11_train_yolov8_run3.py --device 0</code>
Compare all 3 models	<code>python scripts/training/12_compare_all_runs.py</code>
Regenerate this PDF	<code>python scripts/generate_project_pdf.py</code>
Docker (CPU)	<code>docker compose up --build</code>
Docker (GPU)	<code>docker compose -f docker-compose.yml -f docker-compose.gpu.yml up --build</code>

API endpoints (local: <http://127.0.0.1:8000>)

Method	Path	Purpose
GET	<code>/health</code>	Service health + model load status
GET	<code>/models</code>	Model catalog and benchmark metrics
POST	<code>/predict</code>	Single-image inference
POST	<code>/predict/compare</code>	Multi-model side-by-side inference
GET	<code>/docs</code>	Swagger UI (interactive API docs)
GET	<code>/dashboard/</code>	Web UI (upload, webcam, compare)

07. Tech Stack

Layer	Technology	Notes
Language	Python 3.10+	Core runtime
Deep learning	PyTorch	Training & GPU inference
Segmentation	YOLOv5 / YOLO11 / YOLOv8	Instance masks
Annotation	Roboflow	Polygon labels, YOLO export
Computer vision	OpenCV, Pillow	I/O, overlay, encoding
Deduplication	imagehash	Perceptual-hash near-duplicate removal
API framework	FastAPI + Uvicorn	Async REST server
Validation	Pydantic	Request/response schemas
Frontend	HTML / CSS / JavaScript	Dashboard (no framework)
Containers	Docker, Compose	Local + GPU override
Cloud	Render.com	render.yaml + Dockerfile.render
Version control	Git, GitHub	Weights + reports in repo
Testing	httpx	scripts/verify_endpoints.py

Environment variables

Variable	Default	Purpose
HAZARD_DEFAULT_MODEL_ID	yolov5	Production model in API
HAZARD_MODEL_WEIGHTS	Run 4 best_yolov5s.pt	Weight path override
HAZARD_DATASET_ROOT	hazard_dataset_clean/	Training data root
HAZARD_DEVICE	0 / cpu	CUDA device or CPU
HAZARD_IMG_SIZE	640 (416 on Render)	Inference resolution
PORT	8000	API listen port

08. Libraries

Install profiles (pyproject.toml optional extras):

- pip install -e ".[api]" — inference server only
- pip install -e ".[api,run2]" — + Ultralytics (YOLO11 / YOLOv8)
- pip install -e ".[train,api,run2]" — full training + API stack

Library	Category	Purpose in this project
torch / torchvision	ML	Training, GPU inference, tensor ops
ultralytics	ML	YOLO11s and YOLOv8s segmentation
ultralytics/yolov5	ML	YOLOv5s-Seg production inference
fastapi	API	REST endpoints, OpenAPI docs
uvicorn	API	ASGI server
python-multipart	API	File upload handling
opencv-python	CV	Image decode, mask drawing
Pillow	CV	JPEG encoding for API responses
imagehash	Data	Perceptual-hash deduplication
pydantic	API	Schema validation
httpx	Testing	Async HTTP smoke tests
albumentations	Training	Augmentation pipeline
pycocotools	Training	COCO-style mAP evaluation
pyyaml	Config	data.yaml dataset manifests
reportlab	Docs	This PDF generator

09. Project Structure

```
Hazard_Waste_Detection/
|
|-- api/                                FastAPI service (Task 6)
|   |-- main.py                        Routes: /health, /predict, /compare
|   |-- model_manager.py               Model load, cache, routing
|   |-- yolov5_engine.py               YOLOv5s Run 4 inference
|   |-- ultralytics_engine.py          YOLO11 / YOLOv8 inference
|   |-- inference.py                   Image processing, hazard alerts
|   |-- schemas.py                     Pydantic request/response models
|
|-- dashboard/                          Web UI (upload, webcam, compare)
|-- hazard_detection/                   Core library
|   |-- config/                        paths.py, models.py, dataset.py
|
|-- scripts/
|   |-- pipeline/                       00-07 dataset preparation
|   |-- training/                       08-13 train & compare
|   |-- verify_endpoints.py             API smoke tests
|   |-- generate_project_pdf.py          This document generator
|
|-- hazard_dataset_clean/               Frozen dataset (data.yaml in Git)
|-- hazard_dataset/                     Raw merged dataset (local only)
|
|-- runs/
|   |-- yolov5s_run4/                   Production weights & evaluation
|   |-- yolo11s_run2/                   YOLO11 benchmark
|   |-- yolov8s_run3/                   YOLOv8 benchmark
|   |-- comparison/                     3-model JSON / TXT reports
|
|-- docs/                               Reports, guides, this PDF
|-- evaluation_reports/                 Legacy Run 1 evaluation
|-- Dockerfile / Dockerfile.render / render.yaml
|-- run_server.bat / .ps1               Windows launchers
```

10. File Reference

File / path	Description
api/main.py	FastAPI application entry point
api/model_manager.py	Lazy/eager model loading and LRU cache
api/yolov5_engine.py	YOLOv5s-Seg Run 4 inference (torch.load patch)
api/ultralytics_engine.py	YOLO11s / YOLOv8s via Ultralytics API
api/inference.py	Decode uploads, resize, overlay masks, hazard JSON
api/schemas.py	Pydantic models for API I/O
hazard_detection/config/models.py	Model catalog and benchmark metrics
hazard_detection/config/paths.py	Canonical paths (weights, dataset, runs)
hazard_detection/config/dataset.py	CLASS_NAMES, Roboflow source metadata
hazard_detection/data/labels.py	YOLO polygon validation and cleaning
scripts/pipeline/00_build_dataset.py	Merge two Roboflow exports
scripts/pipeline/05_deduplicate_and_resplit.py	Dedup + stratified resplit
scripts/pipeline/06_final_dataset_validation.py	Final QA before training
scripts/training/13_train_yolov5_run4.py	Production YOLOv5s training
scripts/training/12_compare_all_runs.py	3-model validation comparison
scripts/verify_endpoints.py	Automated API endpoint tests
runs/yolov5s_run4/weights/best_yolov5s.pt	Production weights (~15 MB)
runs/comparison/all_runs_comparison.json	Benchmark summary (all models)
hazard_dataset_clean/data.yaml	YOLO training manifest (in Git)
render.yaml	Render.com Blueprint (env vars, Docker)
Dockerfile.render	CPU-optimised image for cloud deploy
README.md	Main documentation (Tasks 1-6, quick start)
docs/PROJECT_PROPOSAL.md	Academic proposal sections 1-11
docs/TASK_DELIVERABLES.md	Per-task deliverable checklist
docs/Hazard_Waste_Detection_Project_Guide.pdf	This document

11. Key Takeaways

Topic	Takeaway
Safety-first design	Two hazard classes with distinct alert types: EXPLOSIVE (Cylinder) and TOXIC (Shock Absorber).
Instance segmentation	Pixel masks outperform bounding boxes in cluttered scrap-yard scenes.
Rigorous dataset engineering	393 unique images, zero leakage, balanced classes — dedup via perceptual hashing.
Three-model benchmark	YOLOv5s (production), YOLO11s, YOLOv8s evaluated on the same frozen split.
Performance target	75-80% precision and recall on validation; Cylinder recall is the primary safety metric.
Production deployment	FastAPI + dashboard deployed on Render; Docker for local/GPU environments.
Privacy by design	Dataset images stay local-only; Git contains manifests, weights, and JSON reports.
Reproducible pipeline	Numbered scripts (00-13) cover annotation through deployment end-to-end.
Extensible architecture	ModelManager routes to YOLOv5 or Ultralytics engines without API contract changes.

End of document — Hazardous Waste Detection — Project Guide
Regenerate: `python scripts/generate_project_pdf.py`